Ahmed *et al. J Big Data* (2020) 7:110
https://doi.org/10.1186/s40537-020-00388-5

Journal of Big Data

RESEARCH Open Access

A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench

N. Ahmed[1*], Andre L. C. Barczak[1], Teo Susnjak[1] and Mohammed A. Rashid[2]

*Correspondence:
nasim751@yahoo.com
[1] School of Natural and Computational Sciences, Massey University, Albany, Auckland 0745, New Zealand
Full list of author information is available at the end of the article

Abstract

Big Data analytics for storing, processing, and analyzing large-scale datasets has become an essential tool for the industry. The advent of distributed computing frameworks such as Hadoop and Spark offers efficient solutions to analyze vast amounts of data. Due to the application programming interface (API) availability and its performance, Spark becomes very popular, even more popular than the MapReduce framework. Both these frameworks have more than 150 parameters, and the combination of these parameters has a massive impact on cluster performance. The default system parameters help the system administrator deploy their system applications without much effort, and they can measure their specific cluster performance with factory-set parameters. However, an open question remains: can new parameter selection improve cluster performance for large datasets? In this regard, this study investigates the most impacting parameters, under resource utilization, input splits, and shuffle, to compare the performance between Hadoop and Spark, using an implemented cluster in our laboratory. We used a trial-and-error approach for tuning these parameters based on a large number of experiments. In order to evaluate the frameworks of comparative analysis, we select two workloads: WordCount and TeraSort. The performance metrics are carried out based on three criteria: execution time, throughput, and speedup. Our experimental results revealed that both system performances heavily depends on input data size and correct parameter selection. The analysis of the results shows that Spark has better performance as compared to Hadoop when data sets are small, achieving up to two times speedup in WordCount workloads and up to 14 times in TeraSort workloads when default parameter values are reconfigured.

Keywords: HiBench, BigData, Hadoop, MapReduce, Benchmark, Spark

Introduction

Hadoop [1] has become a very popular platform in the IT industry and academia for its ability to handle large amounts of data, along with extensive processing and analysis facilities. Different users produce these large datasets, and most of data are unstructured, increasing the requirements for memory and I/O. Besides, the advent of many new applications and technologies brought much larger volumes of complex data, including social media, e.g., Facebook, Twitter, YouTube, online shopping,

machine data, system data, and browsing history [2]. This massive amount of digital data becomes a challenging task for the management to store, process, and analyze.

The conventional database management tools are unable to handle this type of data [3]. Big data technologies, tools, and procedures allowed organizations to capture, process speedily, and analyze large quantities of data and extract appropriate information at a reasonable cost.

Several solutions are available to handle this problems [4]. Distributed computing is one possible solution considered as the most efficient and fault-tolerant method for companies to store and process massive amounts of data. Among this new group of tools, MapReduce and Spark are the most commonly used cluster computing tools. They provide users with various functions using simple application programming interfaces (API). MapReduce is a framework used for distributed computing used for parallel processing and designed purposely to write, read, and process bulky amounts of data [1, 5, 6]. This data processing framework is comprised of three stages: Map phase, Shuffle phase and Reduce phase. In this technique, the large files are divided into several small blocks of equal sizes and distributed across the cluster for storage. MapReduce and Hadoop distributed file systems (HDFS) are core parts of the Hadoop system, so computing and storage work together across all nodes that compose a cluster of computers [7].

Apache Spark is an open-source cluster-computing framework [8]. It is designed based on the Hadoop and its purpose is to build a programing model that "fits a wider class of applications than MapReduce while maintaining the automatic fault tolerance" [9]. It is not only an alternative to the Hadoop framework but it also provides various functions to process real streaming data. Apart from the map and reduce functions, Spark also supports MLib1, GraphX, and Spark streaming for big data analysis. Hadoop MapReduce processing speed is slow because it requires accessing disks for reads and writes. On the other hand, Spark uses memory to store data reducing the read/write cycle [1]. In this paper, we have addressed the above mentioned critical challenges. According to our knowledge, none of the previous works have addressed those challenges. Our proposed work will help the system administrators and researchers to understand the system behavior when processing large scale data sets. The main contributions of this paper are as follows:

- We introduced a comprehensive empirical performance analysis between MapReduce and Spark frameworks by correlating resource utilization, splits size, and shuffle behavior parameters. As per our knowledge, few previous studies have presented information regarding that. Considering this point, the authors have focused on a comprehensive study about various parameters impact with large data set instead of a large number of workloads.
- We accomplished comprehensive comparison work between Hadoop and Spark where large scale datasets (600 GB) are used for the first time. The experiments present the various aspects of cluster performance overhead. We applied two Hibenchmark workloads to test the efficiency of the system under MapReduce and Spark, where the data sets are repeatedly changing.

- We selected several parameters covering different aspects of system behavior. Multiple parameters are used to tune job performance. The results of the analysis will facilitate job performance tuning and enhance the freedom to modify the ideal parameters to enhance job efficiency.
- We measured the scalability of the experiment by repeating the experiment three times, getting the average execution time for each job. Besides, we investigate the system execution time, maximum sustainable throughput and speedup.
- We used a real cluster capable of handling large scale data set (600 GB) with benchmarking tools for a comprehensive evaluation of MapReduce and Spark.

The remainder of the paper is organized as follows: "Related work" section presents a critical review of related research works, and then describes Hadoop and Spark systems. The difference between Hadoop and Spark is explained in "Difference between Hadoop and Spark" section. The experimental setup is presented in "Experimental setup" section. In "The parameters of interest and tuning approach" section, we explain the chosen parameters and tuning approach. "Results and discussion" section presents the performance analysis of the results and finally, we conclude in "Conclusion" section.

Related work

Shi et al. [10] proposed two profiling tools to quantify the performance of the MapReduce and Spark framework based on a micro-benchmark experiment. The comparative study between these frameworks are conducted with batch and iterative jobs. In their work, the authors consider three components: shuffle, executive model, and caching. The workloads, Wordcount, k-means, Sort, Linear Regression, and PageRank, are chosen to evaluate the system behavior based on CPU bound, disk-bound, and network bound [11]. They disabled map and reduce function for all workloads apart of a Sort. For the Sort, the reduce task is configured up to 60 map tasks, and the reduce task conFigured to 120. The map output buffer is allocated to 550 MB to avoid additional spills for sorting the map output. Spark intermediate data are stored in 8 disks where each worker is configured with four threads. The authors claim that Spark is faster than MapReduce when WordCount runs with different data sets (1 GB, 40 GB, and 200 GB). The TeraSort is used by sort-by-key() function. They have found that Spark is faster than MapReduce when the data set is smaller (1 GB), but Mapreduce is nearly two times faster than Spark when the data set is of bigger sizes (40 GB or 100 GB). Besides, Spark is one and a half times faster than MapReduce with machine learning workloads such as K-means and Linear Regression. It is claimed that in a subsequent iteration, Spark is five times faster than MapReduce due to the RDD caching and Spark-GraphX is four times faster than MapReduce.

Li et al. [12] proposed a spark benchmarking suite [13], which significantly enhances the optimization of workload configuration. This work has identified the distinct features of each benchmark application regarding resource consumption, the data flow, and the communication pattern that can impact the job execution time. The applications are characterized based on extensive experiments using synthetic data sets. There are ten different workloads such as Logistic Regression, Support Vector Machine, Matrix Factorization, Page Rank, Tringle Count, SVD++, Hive, RDD Relation, Twitter, and

PageView used with different input data sizes. An eleven nodes virtual cluster is used to analyze the performance of the workloads. The workload analysis is carried out concerning CPU utilization, memory, disk, and network input/output consumption at the time of job execution. They have found that most of the workloads spend more than 50% execution time for MapShuffle-Tasks except logistic regression. They concluded that the job execution time could be reduced while increasing task parallelism to leverage the CPU utilization fully.

Thiruvathukal et al. [14] have considered the importance and implication of the language such as Python and Scala built on the Java Virtual Machine (JVM) to investigate how the individual language affects the systems' overall performance. This work proposed a comprehensive benchmarking test for Massage Passing Interface (MPI) and cloud-based application considering typical parallel analysis. The proposed benchmark techniques are designed to emulate a typical image analysis. Therefore, they presented one mid-size (Argonne Leadership Computing Facility) cluster with 126 nodes, which run on COOLEY [14] and a large scale supercomputer (Cray XC40 supercomputer) cluster with a single node which runs on THETA [14]. Significantly, they have increased some important Spark parameters (Spark driver memory, and executor memory) values as per the machine resource. They have recommended that COOLEY and THETA frameworks are be beneficial for immediate research work and high-performance computing (HPC) environments.

Marcue et al. [15] present the comparative analysis between Spark and Flink frameworks for large scale data analysis. This work proposed a new methodology for iterative workloads (K-Means, and Page Rank) and batch processing workloads (WordCount, Grep, and TeraSort) benchmarking. They considered four most important parameters that impact scalability, resource consumption, and execution time. Grid 5000 [16] has used upto 100 nodes cluster deploying Spark and Flink. They have recommended that Spark parameter (i.e., parallelism and partitions) configuration is sensitive and depends on data sets, while the Flink is highly extensive memory oriented.

Samadi et al. [7] has investigated the criteria of the performance comparison between Hadoop and Spark framework. In his work, for an impartial comparison, the input data size and configuration remained the same. Their experiment used eight benchmarks of the HiBench suite [13]. The input data was generated automatically for every case and size, and the computation was performed several times to find out the execution time and throughput. When they deployed microbenchmark (Short and TeraSort) on both systems, Spark showed higher involvement of processor in I/Os while Hadoop mostly processed user tasks. On the other hand, Spark's performance was excellent when dealing with small input sizes, such as micro and web search (Page Rank). Finally, they concluded that Spark is faster and very strong for processing data in-memory while Hadoop MapReduce performs maps and reduces function in the disk.

In another paper, Samadi et al. [9] proposed a virtual machine based on Hadoop and Spark to get the benefit of virtualization. This virtual machine's main advantage is that it can perform all operations even if the hardware fails. In this deployment, they have used Centos operating system built a Hadoop cluster based on a pseudo-distribution mode with various workloads. In their experiments, they have deployed the Hadoop machine on a single workstation and all other demos on its JVM. To justify the big data

framework, they have presented the results of Hadoop deployment on Amazon Elastic Computing (EC2). They have concluded that Hadoop is a better choice because Spark requires more memory resources than Hadoop. Finally, they have suggested that the cluster configuration is essential to reduce job execution time, and the cluster parameter configuration must align with Mappers and Reducers.

The computational frameworks, namely Apache Hadoop and Apache Spark, were investigated by [17]. In this investigation, the Apache webserver log file was taken into consideration to evaluate the two frameworks' comparative performance. In these experiments, they have used Okeanos's virtualized computing resources based on infrastructures as a Service (IaaS) developed by the Greek Research and Technology Network [17]. They proposed a number of applications and conducted several experiments to determine each application's execution time. They have used various input files and the slave nodes to find out the execution time. They have found that the execution time is proportional to the input data size. They have concluded that the performance of Spark is much better in most cases as compared to Hadoop.

Satish and Rohan [18] have shown a comparative performance study between Hadoop MapReduce and Spark-based on the K-means algorithm. In this study, they have used a specific data set that supports this algorithm and considered both single and double nodes when gathering each experiment's execution time. They have concluded that the Spark speed reaches up to three times higher than the MapReduce, though Spark performance heavily depends on sufficient memory size [19].

Lin et al. [20] have proposed a unified cloud platform, including batch processing ability over standalone log analysis tools. This investigation has considered four different frameworks: Hadoop, Spark, and warehouse data analysis tools Hive and Shark. They implemented two machine learning algorithms (K-means and PageRank) based on this framework with six nodes to validate the cloud platform. They have used different data sizes as inputs. In the case of K-means, as the data size increased and exceed memory size, the latency schedule and overall Spark performance degraded. However, the overall performance was still six times higher than Hadoop on average. On the other hand, Shark shows significant performance improvement while using queries directly from disk.

Petridis et al. [21] have investigated the most important Spark parameters shown in Table 4 and given a guideline to the developers and system administrators to select the correct parameter values by replacing the default parameter values based on trial-and-error methodology. Three types of case studies with different categories such as Shuffle Behavior, Compression and Serialization, and Memory Management parameters were performed in this study. They have highlighted the impact of memory allocation and serialization when the number of cores and default parallelism values change. Therefore, there are 12 parameters chosen with three benchmarking applications: sort-by-key, shuffling, and k-means. The sort-by-key experiments used both 1 million and 1 billion key-values of lengths 10 and 90 bytes and the optimal degree of partition is set to 640. The Hash performance is increased to 127 s, which is 30 s faster than the default parameter, and shuffle.file.buffer is increased by 140 s. The rest of the parameters do not play any important role in improving the performance. For another Shuffling experiment, they used a 400 GB dataset. The Hash shuffle performance is degraded by 200 s, and

Tungsten-Sort speed is increased by 90 s. By decreasing the buffer size from 32 to 15 KB, the system performance was degraded by about 135s, which is more than 10% from the primary selection. For K-means, they used two sizes of data input (100 MB and 200 MB). They have not found significant k-means performance improvement by changing the parameters. Therefore, they have concluded that based on their methodology, the speedup achievement is tenfold. However, the main challenges of tuning Hadoop and Spark configuration parameters are due to the complicated behavior of distributed large scale systems while the parameter selection is not always trivial for the system administrators. Inappropriate combination of parameter values can affect the overall system performance. Inappropriate combination of parameter values can affect the overall system performance.

The published literature in Table 1 presents some empirical studies. None of these studies have considered larger data sizes (600 GB), more parameters, and real clusters. In our study, we chose a conventional trial-and-error approach [21], larger data set, and 18 important parameters (listed in Tables 3 and 4) from resource utilization, input splits, and shuffle category.

Difference between Hadoop and Spark

Hadoop [22] is a very popular and useful open-source software framework that enables distributed storage, including the capability of storing a large amount of big datasets across clusters. It is designed in such a way that it can scale up from a single server to thousands of nodes. Hadoop processes large data concurrently and produces fast

Table 1 Published related work

Author's	Date	Workloads	Data size	Parameters	Hardware
Lin et al. [20]	2013	K-means PageRank	10,000 to 20 mil points 1 mil to 10 mil points	Log analysis	Nodes—6, 2 CPU cores 4 GB memory per node Nodes—4, 16 CPU cores 48 GB memory per node
Satish and Rohan [18]	2015	K-means	62–1240 MB	Default	Virtual machine Nodes—2, 4 GB RAM and 500 GB (HD)
Yasir Samadi et al. [7]	2016	Micro Benchmarks Web Search SQL Machine Learning	18–328 MB 5000 to 12 * 10e4 pages	3	Virtual machine Disk (SDD)—40 GB
Petridis et al. [21]	2017	K-means shuffling and sort-by-Key	400 GB	12	Barcelona supercomputing center
Mavridis et al. [17]	2017	Spark SQL and Spark Hive	1.1 GB, 1.5 GB and 11 GB	Log analysis	Virtual machine—6 Memory—8 GB Master node—8 cores Salve node—4 cores
Yasir Samadi et al. [9]	2018	Micro Benchmarks Web Search SQL Machine Learning	1 GB, 5 GB and 8 GB	3	Virtual machine Disk(SDD)—40 GB
Proposed experiments	2020	WordCount and TeraSort	50–600 GB	18	SNCC, Production Cluster CPU cores—80 Total Storage—60 TB Master node—1 Slaves nodes—9

results. With Hadoop, the core parts are Hadoop Distributed File System (HDFS) and MapReduce.

HDFS [23] splits the files into small pieces into blocks and saves them into different nodes. There are two kinds of nodes on HDFS: data-nodes (worker) and name-nodes (master nodes) [24, 25]. All the operations, including delete, read, and write, are based on these two types of nodes. The workflow of HDFS is like the following flow: firstly, the name-node asks for access permission. If accepted, it will turn the file name into a list of HDFS block IDs, including the files and the data-nodes that saved the blocks related to that file. The ID list will then be sent back to the client, and the users can do further operations based on that.

MapReduce [26] is a computing framework that includes two operations: Mappers and Reducers. The mappers will process files based on the map function and transfer them into the new key-value pairs [27]. Next, the new key-value pairs are assigned to different partitions and sorted based on their keys. The combiner is optional and can be recognized as a local reduces operation which allows counting the values with the same key in advance to reduce the I/O pressure. Finally, partitions will divide the intermediate key-value pairs into different pieces and transfer them to a reducer. MapReduce needs to implement one operation: shuffle. Shuffle means transferring the mapper output data to the proper reducer. After the shuffle process is finished, the reducer starts some copy threads (Fetcher) and obtains the output files of the map task through HTTP [28]. The next step is merging the output into different final files, which are then recognized as reducer input data. After that, the reducer processes the data based on the reduced function and writes the output back to the HDFS. Figure 1 depicts a Hadoop MapReduce architecture.

Spark became an open-source project from 2010. Zahari has developed this project at UC Berkely's AMPLab in 2009 [4, 29]. Spark offers numerous advantages for developers to build big data applications. Spark proposed two important terms: Resilient Distributed Datasets (RDD) and Directed Acyclic Graph (DAG). These two techniques work together perfectly and accelerate Spark up to tens of times faster than Hadoop under certain circumstances, even though it usually only achieves a performance two to three times more quickly than MapReduce. It supports multiple sources that have a fault tolerance mechanism that can be cached and supports parallel operations. Besides, it can represent a single dataset with multiple partitions. When Spark runs on the Hadoop cluster, RDDs will be created on the HDFS in many formats supported by Hadoop, likewise text and sequence files. The DAG scheduler [30] system expresses the dependencies of RDDs. Each spark job will create a DAG and the scheduler will drive the graph into the different stages of tasks then the tasks will be launched to the cluster. The DAG will be created in both maps and reduce stages to express the dependencies fully. Figure 2 illustrates the iterative operation on RDD. Theoretically, limited Spark memory causes the performance to slow down.

Experimental setup

Cluster architecture

In the last couple of years, many proposals came from different research groups about the suitability of Hadoop and Spark frameworks when various types of data

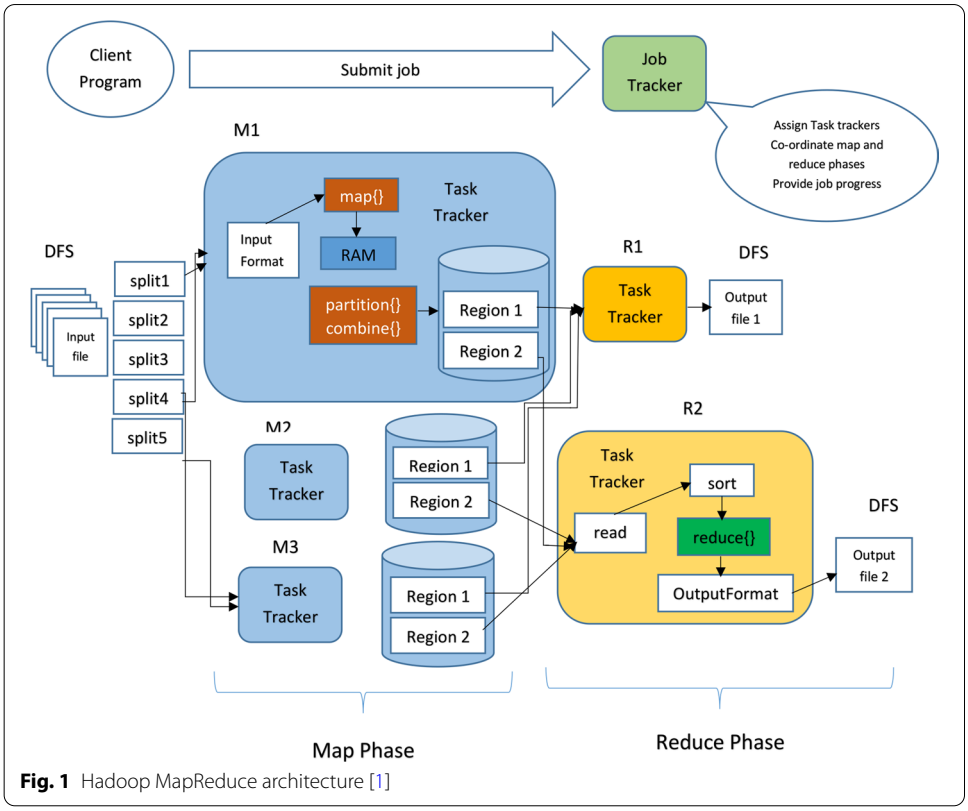

Fig. 1 Hadoop MapReduce architecture [1]

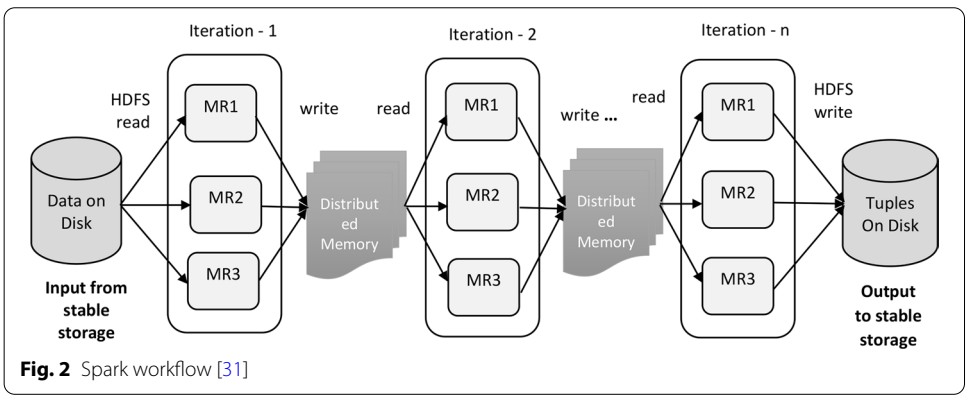

Fig. 2 Spark workflow [31]

of different sizes are used as input in different clusters. Therefore, it becomes necessary to study the performance of the frameworks and understand the influence of various parameters. For the experiments, we will present our cluster performance based on MapReduce and Spark using the HiBench suite [23, 23]. In particular, we have selected two Hibench workloads out of thirteen standard workloads to represent the two types of jobs, namely WordCount (aggregation job) [32], and TeraSort (shuffle job) [33] with large datasets. We selected both the workloads because of their complex characteristics to study how efficiently both the workloads analyze the cluster performance by correlating MapReduce and Spark function with a combination of groups of parameters.

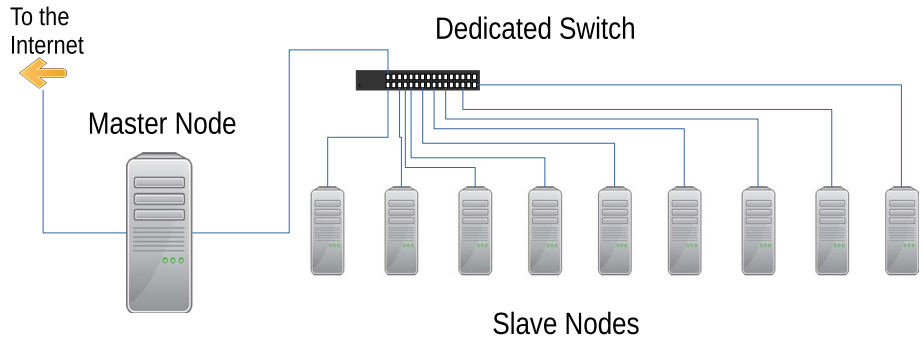

Fig. 3 Hadoop cluster nodes

Table 2 Experimental Hadoop cluster

Server configuration	Processor	2.9 GHz
	Main Memory	64 GB
	Local Storage	10 TB
Node configuration	CPU	Intel(R) Xeon(R) CPU E3-1231 v3 @ 3.40GHz
	Main Memory	32 GB
	Number of Nodes	10
	Local Storage	6 TB each, 60 TB total
	CPU cores	8 each, 80 total
Software	Operating System	Ubuntu 16.04.2 (GNU/Linux 4.13.0-37-generic×86 64)
	JDK	1.7.0
	Hadoop	2.4.0
	Spark	2.1.0
Workload	Micro Benchmarks	WordCount, and TeraSort

Hardware and software specification

The experiments were deployed in our own cluster. The cluster is configured with 1 master and 9 slaves nodes which is presented in Fig. 3. The cluster has 80 CPU cores and 60 TB local storage. The implemented hardware is suitable for handling various difficult situations in Spark and MapReduce.

The detailed Hadoop cluster and software specifications are presented in Table 2. All our jobs run in Spark and MapReduce. We have selected Yarn as a resource manager, which can help us monitor each working node's situation and track the details of each job with its history. We have used *Apache Ambari* to monitor and profile the selective workloads running on Spark and MapReduce. It supports most of the Hadoop components, including HDFS, MapReduce, Hive, Pig, Hbase, Zookeeper, Sqoop, and Hcatalog" [34]. Besides, Ambari supports the user to control the Hadoop cluster on three aspects, namely provision, management, and monitoring.

Table 3 Hadoop configuration parameters

Configuration parameters category	Hadoop	Tuned values
Resource utilization	mapreduce.reduce.memory	8 GB
	mapred.reduce.task	16,384 MB, 25,600 MB
	mapreduce.reduce.cpu.vcores	4
Input split	mapred.min.split.size, mapred.max.split.size	128 MB (default), 256 MB, 512 MB, 1024 MB
Shuffle	i/o.sort.mb	25, 50, 75, 100
	i/o.sort.factor	512, 1024, 1536, 2047
	mapreduce.reduce.shuffle.parallelcopies	50, 100, 150, 200
	mapreduce.task.io.sort.factor	15, 30, 45, 60

Table 4 Spark configuration parameters

Configuration parameters category	Spark	Tuned values
Resource utilization	num-executors	50
	executor-cores	4
	executor-memory	8 GB
Input split	spark.hadoop.MapReduce.input .fileinputformat.split.minsize	128 MB (default), 256MB, 512MB, 1024MB
Shuffle	spark.shuffle.file.buffer	16 k, 32 k (default), 48 k, 64 k
	spark.reducer.maxSizeInFlight	32 M, 48 M (default), 64 M, 96 M
	spark.hadoop.dfs.replication	1
	spark.default.parallelism	80, 100, 200, 300

Workloads

As stated above, in this study we chose two workloads for the experiments [32, 33]:

WordCount: The wordCount workload is map-dependent, and it counts the number of occurrences of separate words from text or sequence file. The input data is produced by *RandomTextWriter*. It splits into each word by using the map function and generates intermediate data for the reduce function as a key-value [35]. The intermediate results are added up, generating the final word count by the reduce function.

TeraSort: The TeraSort package was released by Hadoop in 2008 [36] to measure the capabilities of cluster performance. The input data is generated by the *TeraGen* function which is implemented in Java. The TeraSort function does the sorting using the MapReduce, and the TeraValidate function is used to validate the output of the sorted data. For both workloads, we used up to 600 GB of synthetic input data generated using a string concatenation technique.

The parameters of interest and tuning approach

Tuning parameters in Apache Hadoop and Apache Spark is a challenging task. We want to find out which parameters have important impacts on system performance. The configuration of the parameters needs to be investigated according to work-load,

data size, and cluster architecture. We have conducted a number of experiments using Apache Hadoop and Apache Spark with different parameter settings. For this experiment, we have chosen the core MapReduce and Spark parameter setting from resource utilization, input splits and shuffle groups. The selected tuned parameters with their respective tuned values on the map-reduce and Spark category are shown in Tables 3 and 4.

Results and discussion

In this section, the results obtained after running the jobs are evaluated. We have used synthetic input data and used the same parameter configuration for a realistic comparison. Each test was repeated three times, and the average runtime was plotted in each graph. For both frameworks, we show the execution time, throughput, and speedup to compare the two frameworks and visualize the effects of changing the default parameters.

Execution time

The execution time is affected by the input data sizes, the number of active nodes, and the application types. We have fixed the same parameters for the fair comparative analysis, such as the number of executors to 50, executor memory to 8 GB, executor cores to 4.

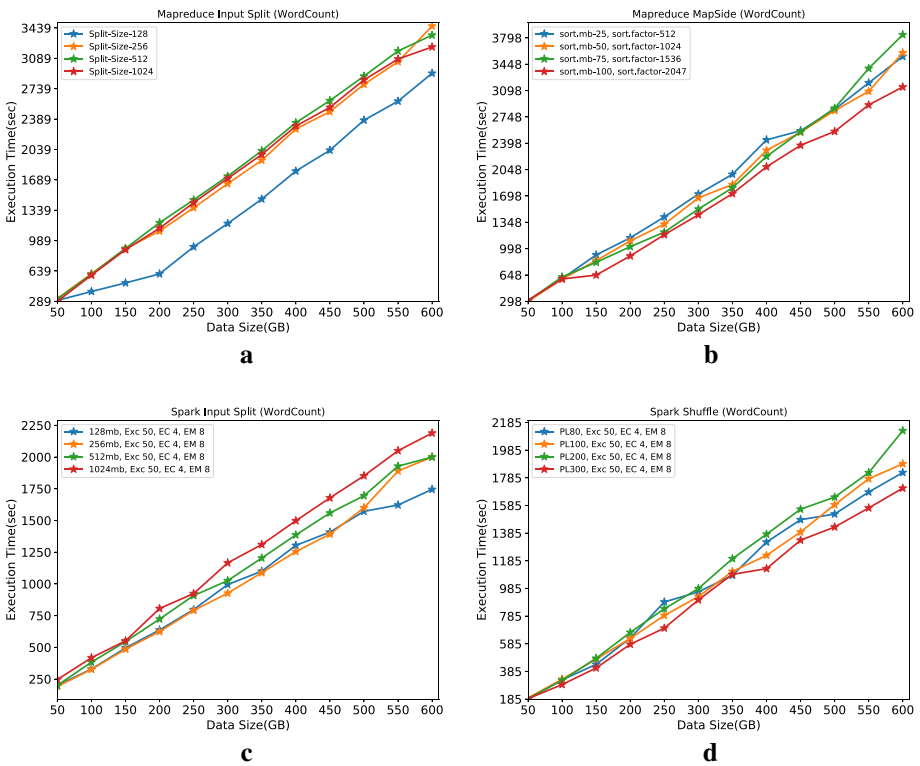

Fig. 4 The performance of the WordCount application with a varied number of input splits and shuffle tasks

Figure 4a, b show how MapReduce and Spark execution time depend on the datasets' size and the different input splits and shuffle parameters. The execution time of MapReduce WordCount workload with the default input split size (128 MB) and shuffle parameter (*sort.mb* 100, *sort.factor* 2047) obtained better execution time for entire data sizes compared to other parameters. Hadoop Map and Reduce function behave better because of their faster execution time and overlooked container initialization overhead for specific workload types. This result suggests that the default parameter is more suitable for our cluster when using data sizes from 50 to 600 GB.

In Fig. 4c the default input splits of Spark is 128 MB. Previously, we have mentioned that the number of executors, executor memory, and executor cores are fixed. From the above Fig. 4c, we see that the execution time of input split size 256 MB outperforms the default set up until 450 GB data sizes. In fact, the default splits size (128 MB) is more efficient when the data size is larger than the 450 GB. Notably, we can see that the default parameter shows better execution performance when the data set reaches 500 GB or above. The new parameter values can improve the processing efficiency by 2.2% higher than the default value (128 MB). Table 5 presents the experimental data of WordCount workload between MapReduce and Spark while the default parameters are changing.

For the Spark shuffle parameter, we have chosen the default serializer, the *(JavaSerializer)* because of the simplicity and easy control of the performance of the serialization [37]. In this category, the serializer is PL100 object [37]. We can see from Fig. 4d that the improvement rate is significantly increased when we set the PL value to 300. It is evident that the best performance is achieved for sizes larger than 400 GB. Also, it shows that when tuning the PL value to 300, the system can achieve a 3% higher improvement for the rest of the data sizes. Consequently, we can conclude that input splits can be considered an important factor in enhancing Spark WordCount jobs' efficiency when executing small datasets.

Figure 5a is comparing MapReduce TeraSort workloads based on input splits that include default parameters. In this analysis, we have set (*Red_Task* and *InSp*) value fixed with default split size 128 MB. We have changed the parameter values and tested whether the splits' size can keep the impact on the runtime. So, for this reason, we have selected three different sizes: 256 MB, 512 MB, and 1024 MB. We have observed that with a split size of 256MB, the execution performance is increased by around 2% in datasets with up to 300 GB. On the contrary, when the data sizes are larger than 300 GB, the default size outperforms split size equals 512 MB. Moreover, we have noticed that the improvement rates are similar when the data sizes are smaller than 200 GB.

Table 5 The best execution time of MapReduce and Spark with WordCount workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (WordCount)	128	2376
Spark input splits (WordCount)	256	1392
MapReduce shuffle (WordCount)	100	2371
Spark shuffle (WordCount)	300	1334

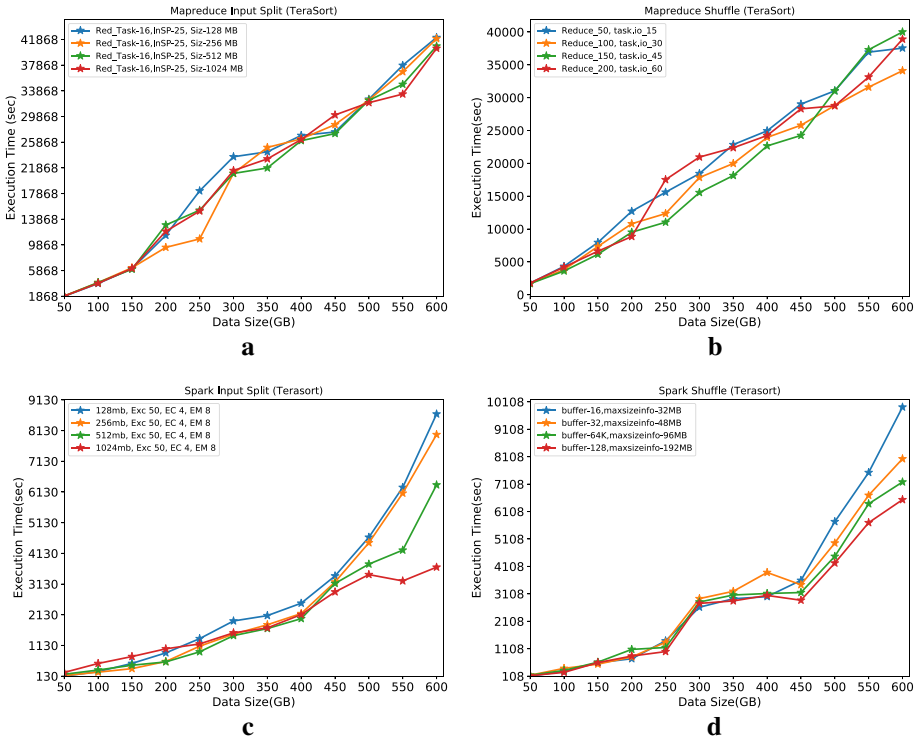

Fig. 5 The performance of the TeraSort application with a varied number of input splits and shuffle tasks

Figure 5b illustrates the execution performance with the MapReduce shuffle parameter for the TeraSort workload. We have seen that the average execution time behaves linearly for sizes up to 450 GB when the parameter change to (*Reduce_150* and *task.io_45*) as compared to the default configuration (*Reduce_100* and *task.io_30*). Besides, We have also noticed that the default configuration is outperforming all other settings when the data sizes are larger than 450 GB. So, we can conclude that by changing the shuffled value, the system execution performance improves by 1%. In general, this is very unlikely that the default size has optimum performance for larger data sizes.

Figure 5c illustrates the Spark input split parameter execution performance analysis for the TeraSort workload. The Spark executor memory, number of executors, and executor memory are fixed while changing the block size to measure the execution performance. Apart from the default block size (128 MB), there are 3 pairs (256 MB, 512 MB, and 1024 MB) of block size is taken into this consideration. Our results revealed that the block size 512 MB and 1024 MB present better runtime for sizes up to 500 GB data size. We have also observed a significant performance improvement achieved by the 1024 block size, which is 4% when the data size is larger than 500 GB. Thus, we can conclude that by adding the input splits block size for large scale data size, Spark performance can be increased.

Figure 5d shows Spark shuffle behaviour performance for TeraSort workloads. We have taken two important default parameters (*buffer* = 32, *spark.reducer.maxSizeIn Flight* = 48 MB) into our analysis. We have found that when the buffer and *maxSizeIn-Flight* are increased by 128 and 192, the execution performance increased proportionally

Table 6 The best execution time of MapReduce and Spark with Terasort workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (TeraSort)	256	21,014
Spark input splits (TeraSort)	512 & 1024	3780 & 3439
MapReduce shuffle (TeraSort)	150 & 45	24,250
Spark shuffle (TeraSort)	128 & 192	6540

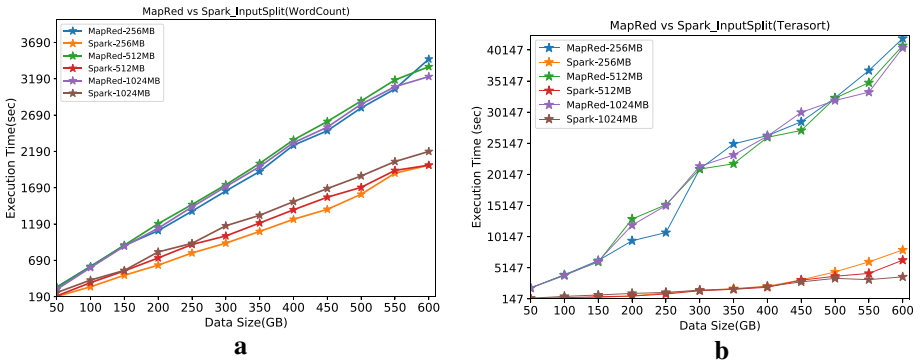

Fig. 6 The comparison of Hadoop and Spark with WordCount and TeraSort workload with varied input splits and shuffle tasks

up to 600 GB data sizes. Our results show that the default execution is equal, with a tested value of up to 200 GB data sizes. The possible reason for this performance improvement is the larger number of splits size for different executors. Table 6 presents the experimental data of the TeraSort workload between MapReduce and Spark, while the default parameters are changing.

Figure 6a illustrates the comparison between Spark and MapReduce for WordCount and TeraSort workloads after applying the different input splits. We have observed that Spark with WordCount workloads shows higher execution performance by more than 2 times when data sizes are larger than 300 GB for WordCount workloads. For the smaller data sizes, the performance improvement gap is around ten times. Figure 6 shows a TeraSort workload for MapReduce and Spark. We can see that Spark execution performance is linear and proportionally larger as the data size increase. Also, we noticed that the runtime for MapReduce jobs are not as linear in relation to the data size as Spark jobs. The possible reason could be unavoidable job action on the clusters and as a result that the dataset is larger than the available RAM. So, we conclude that MapReduce has slower data sharing capabilities and a longer time to the read-write operation than Spark [4].

Throughput

The throughput metrics are all in MB per second. For this analysis, we only considered the best results from each category. We have observed that MapReduce throughput performance for the TeraSort workload is decreasing slightly as the data size crosses beyond 200 GB. Besides, for the WordCount workload, the MapReduce throughput is almost linear. For the Spark TeraSort workload, it can be observed that

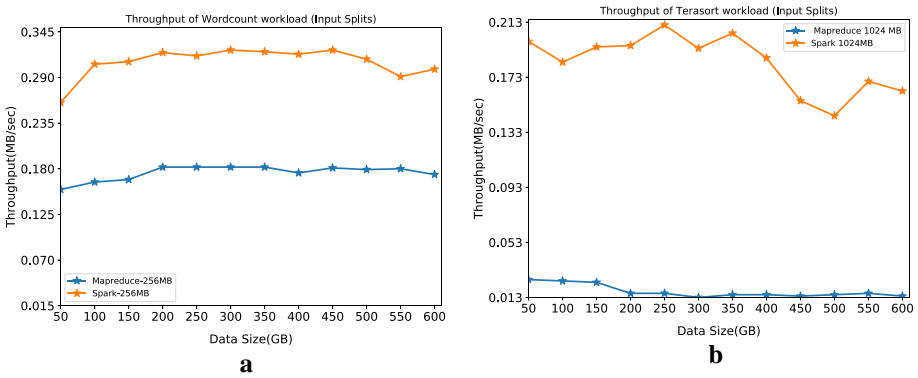

Fig. 7 Throughput of WordCount and TeraSort workload

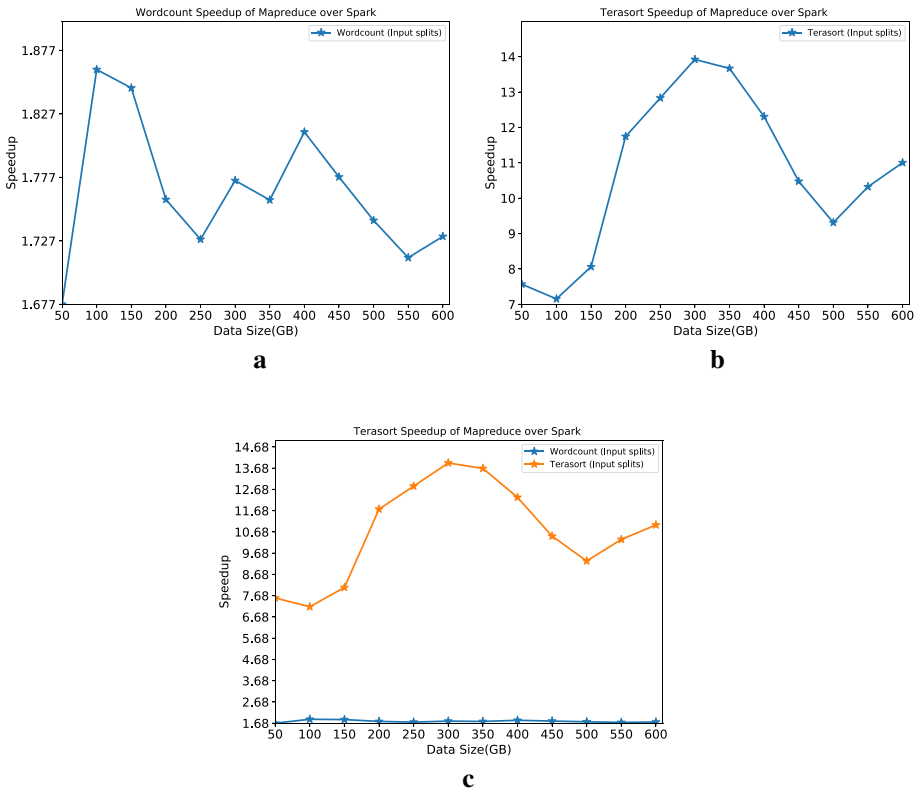

Fig. 8 Spark over MapReduce speedup on input splits and shuffle

the throughput is not constant, but for the WordCount workload, the throughput is almost constant. In this analysis, the main focus was to present the throughput difference between WordCount and TeraSort workload for MapReduce and Spark. We found that WordCount workload remains almost stable for most of the data sizes, and concerning the TeraSort workload, MapReduce remain stable than Spark (see Fig. 7).

Speedup

Figure 8a–c show the Spark's speed up compared to MapReduce. Figure 8a, b depicts individual workload speedup. The best results are taken into this consideration from each category in order to get a speedup. From the above figures, we can see that as the data size increases, WordCount workload speedup decreases with some non-linearity. Besides, we can see that the TeraSort speedup decreases when data reaches sizes larger than 300 GB. Notably, as the data size increases to more than 500GB for both workloads, the speedup starts to increase. Figure 8c illustrates the speedup comparison between the workloads. It can be seen that the TeraSort workload outperforms WordCount workload and achieves an all-time maximum speedup of around 14 times. The literature presents that Spark is up to ten times faster than Hadoop under certain circumstances and in normal conditions, and it only achieves a performance two to three times faster than MapReduce [38]. However, this study found that Spark performance is degraded when the input data size is big.

Conclusion

This article presented the empirical performance analysis between Hadoop and Spark based on a large scale dataset. We have executed WordCount and Terasort workloads and 18 different parameter values by replacing them with default set-up. To investigate the execution performance, we have used trial-and-error approach for tuning these parameters performing number of experiments on nine node cluster with a capacity of 600 GB dataset. Our experimental results confirm that both Hadoop and Spark systems performance heavily depends on input data size and right parameter selection and tuning. We have found that Spark has better performance as compared to Hadoop by two times with WordCount work load and 14 times with Tera-Sort workloads respectively when default parameters are tuned with new values. Further more, the throughput and speedup results show that Spark is more stable and faster than Hadoop because of Spark data processing ability in memory instead of store in disk for the map and reduced function. We have also found that Spark performance degraded when input data was larger.

As future work, we plan to add and investigate 15 HiBench workloads, consider more parameters under resource utilization, parallelization, and other aspects, including practical data sets. The main focus would be to analyze the job performance based on auto-tuning techniques for MapReduce and Spark when several parameter configurations replace the default values.

Acknowledgements
The authors acknowledge Sibgat Bazai for his valuable suggestions.

Authors' contributions
NA was the main contributor of this work. He has done an initial literature review, data collection, experiments, prepare results, and drafted the manuscript. ALCB and TS deployed and configured the physical Hadoop cluster. ALCB also worked closely with NA to review, analyze, and manuscript preparation. TS and MAR helped to improve the final paper. All authors read and approved the final manuscript.

Funding
This work was not funded.

Availability of data and materials
The data that support the findings of this study are available from the corresponding author upon reasonable request.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare that they have no competing interests.

Author details
[1] School of Natural and Computational Sciences, Massey University, Albany, Auckland 0745, New Zealand. [2] Department of Mechanical and Electrical Engineering, Massey University, Auckland 0745, New Zealand.

Received: 30 July 2020 Accepted: 26 November 2020
Published online: 14 December 2020

References

1. Apache Hadoop Documentation 2014. http://hadoop.apache.org/. Accessed 15 July 2020.
2. Verma A, Mansuri AH, Jain N. Big data management processing with hadoop mapreduce and spark technology: A comparison. In: 2016 symposium on colossal data analysis and networking (CDAN). New York: IEEE; 2016. p. 1–4.
3. Management Association IR. Big Data: concepts, methodologies, tools, and applications. Hershey: IGI Global; 2016.
4. Zaharia M, Chowdhury M, Das T, Dave A, Ma J, Mccauley M, Franklin M, Shenker S, Stoica I. Fast and interactive analytics over hadoop data with spark. Usenix Login. 2012;37:45–51.
5. Dean J, Ghemawat S. Mapreduce: simplified data processing on large clusters. Commun ACM. 2008;51(1):107–13.
6. Wang G, Butt AR, Pandey P, Gupta K. Using realistic simulation for performance analysis of mapreduce setups. In: Proceedings of the 1st ACM workshop on large-scale system and application performance; 2009. p. 19–26.
7. Samadi Y, Zbakh M, Tadonki C. Comparative study between hadoop and spark based on hibench benchmarks. In: 2016 2nd international conference on cloud computing technologies and applications (CloudTech). New York: IEEE; 2016. p. 267–75.
8. Ahmadvand H, Goudarzi M, Foroutan F. Gapprox: using gallup approach for approximation in big data processing. J Big Data. 2019;6(1):20.
9. Samadi Y, Zbakh M, Tadonki C. Performance comparison between hadoop and spark frameworks using hibench benchmarks. Concurr Comput Pract Exp. 2018;30(12):4367.
10. Shi J, Qiu Y, Minhas UF, Jiao L, Wang C, Reinwald B, Özcan F. Clash of the titans: mapreduce vs. spark for large scale data analytics. Proc VLDB Endow. 2015;8(13):2110–211.
11. Veiga J, Expósito RR, Pardo XC, Taboada GL, Tourifio J. Performance evaluation of big data frameworks for large-scale data analytics. In: 2016 ieee international conference on Big Data (Big Data). New York: IEEE; 2016. p. 424–31.
12. Li M, Tan J, Wang Y, Zhang L, Salapura V. Sparkbench: a comprehensive benchmarking suite for in memory data analytic platform spark. In: Proceedings of the 12th ACM international conference on computing frontiers; 2015. p. 1–8.
13. Wang L, Zhan J, Luo C, Zhu Y, Yang Q, He Y, Gao W, Jia Z, Shi Y, Zhang S. Bigdatabench: a big data benchmark suite from internet services. In: 2014 IEEE 20th international symposium on high performance computer architecture (HPCA). New York: IEEE; 2014. p. 488–99.
14. Thiruvathukal GK, Christensen C, Jin X, Tessier F, Vishwanath V. A benchmarking study to evaluate apache spark on large-scale supercomputers. 2019; arXiv preprint arXiv:1904.11812.
15. Marcu O-C, Costan A, Antoniu G, Pérez-Hernández MS. Spark versus flink: Understanding performance in big data analytics frameworks. In: 2016 IEEE international conference on cluster computing (CLUSTER). New York: IEEE; 2016. p. 433–42.
16. Bolze R, Cappello F, Caron E, Daydé M, Desprez F, Jeannot E, Jégou Y, Lanteri S, Leduc J, Melab N, et al. Grid'5000: a large scale and highly reconfigurable experimental grid testbed. Int J High Perform Comput Appl. 2006;20(4):481–94.
17. Mavridis I, Karatza E. Log file analysis in cloud with apache hadoop and apache spark 2015.
18. Gopalani S, Arora R. Comparing apache spark and map reduce with performance analysis using k-means. Int J Comput Appl. 2015;113(1):8–11.
19. Gu L, Li H. Memory or time: Performance evaluation for iterative operation on hadoop and spark. In: 2013 IEEE 10th international conference on high performance computing and communications & 2013 IEEE international conference on embedded and ubiquitous computing. New York: IEEE; 2013. p. 721–7.
20. Lin X, Wang P, Wu B. Log analysis in cloud computing environment with hadoop and spark. In: 2013 5th IEEE international conference on broadband network & multimedia technology. New York: IEEE; 2013. p. 273–6.
21. Petridis P, Gounaris A, Torres J. Spark parameter tuning via trial-and-error. In: INNS conference on big data. Berlin: Springer; 2016. p. 226–37.
22. Landset S, Khoshgoftaar TM, Richter AN, Hasanin T. A survey of open source tools for machine learning with big data in the hadoop ecosystem. J Big Data. 2015;2(1):24.
23. HiBench Benchmark Suite. https://github.com/intel-hadoop/HiBench. Accessed 15 July 2020.
24. Shvachko K, Kuang H, Radia S, Chansler R. The hadoop distributed file system. In: 2010 IEEE 26th symposium on mass storage systems and technologies (MSST). New York: IEEE; 2010. p. 1–10.
25. Luo M, Yokota H. Comparing hadoop and fat-btree based access method for small file i/o applications. In: International conference on web-age information management. Berlin: Springer; 2010. p. 182–93.
26. Taylor RC. An overview of the hadoop/mapreduce/hbase framework and its current applications in bioinformatics. BMC Bioinform. 2010;11:1.

27. Vohra D. Practical Hadoop ecosystem: a definitive guide to hadoop-related frameworks and tools. California: Apress; 2016.
28. Lee K-H, Lee Y-J, Choi H, Chung YD, Moon B. Parallel data processing with mapreduce: a survey. AcM sIGMoD record. 2012;40(4):11–20.
29. Zaharia M, Chowdhury M, Franklin MJ, Shenker S, Stoica I. Spark: cluster computing with working sets. HotCloud. 2010;10:95.
30. Kannan P. Beyond hadoop mapreduce apache tez and apache spark. San Jose State University); 2015. http://www.sjsu.edu/people/robert.chun/courses/CS259Fall2013/s3/F.pdf. Accessed 15 July 2020.
31. Spark Core Programming. https://www.tutorialspoint.com/apache_spark/apache_spark_rdd.htm. Accessed 15 July 2020.
32. Huang S, Huang J, Dai J, Xie T, Huang B. The hibench benchmark suite: Characterization of the mapreduce-based data analysis. In: 2010 IEEE 26th international conference on data engineering workshops (ICDEW 2010). New York: IEEE; 2010. p. 41–51.
33. Chen C-O, Zhuo Y-Q, Yeh C-C, Lin C-M, Liao S-W. Machine learning-based configuration parameter tuning on hadoop system. In: 2015 IEEE international congress on big data. New York: IEEE; 2015. p. 386–92.
34. Ambari. https://ambari.apache.org/. Accessed 15 July 2020.
35. Xiang L-H, Miao L, Zhang D-F, Chen F-P. Benefit of compression in hadoop: A case study of improving io performance on hadoop. In: Proceedings of the 6th international asia conference on industrial engineering and management innovation. Berlin: Springer; 2016. p. 879–90.
36. O'Malley O. Terabyte sort on apache hadoop. Report, Yahoo!; 2008. http://sortbenchmark.org/YahooHadoop.pdf. Accessed 15 July 2020.
37. Apache Tuning Spark 1.1.1. https://spark.apache.org/docs/1.1.1/tuning.html. Accessed 15 July 2020.
38. Rathore MM, Son H, Ahmad A, Paul A, Jeon G. Real-time big data stream processing using gpu with spark over hadoop ecosystem. Int J Parallel Progr. 2018;46(3):630–46.

Publisher's Note